

Architecture Handbook

How LAMA is built and how to operate it — the five-stage pipeline, the technology behind each layer, and the technical architecture of the runtime, the model fabric, the knowledge base and the agentic code generator.

PART I

The System

What LAMA does, the stack it runs on, and how the pieces fit together.

PART II

The Internals

Model fabric, knowledge base, agent pipeline, data model and API surface.

01 What LAMA is

LAMA — Legacy Application Modernization & Alignment is a full-stack, AI-assisted migration studio. You point it at a folder of legacy code — PHP/CodeIgniter, JSP, .NET, Python, JavaScript — and it walks that application through a deterministic five-stage pipeline, producing freezable, GitHub-pushable artifacts at every stage.

The reference pilot is the **PMIS Migration Pilot**: PHP 8 / CodeIgniter 4 / MariaDB → FastAPI / Python 3.12 / PostgreSQL.

The five stages

#	Stage	Produces	Freeze unlocks
1	Discovery	Knowledge base (YAML/TOON), IEEE-830 SRS (12 sections), business ontology	DataModel
2	DataModel	OLTP DDL, OLAP DDL, Kimball bus matrix, 3 migration scripts	Architecture
3	Architecture	Service map, HLD, LLD, API contracts, sequence diagrams	CodeGen
4	CodeGen	Per-service source tree, Dockerfiles, ZIP, GitHub push	Living
5	Living	Selenium tests, SRS-drift detection, runtime observability	—

Stages are strictly sequential. Every stage-N+1 route calls `pipeline.require_stage_context(...)` and returns HTTP 400 if the upstream stage is not frozen. The only sanctioned bypass is marking an intermediate stage *skipped*, which writes a minimal StageContext so the downstream gate still passes.

Two tracks beside the pipeline

Not all work flows through the five stages. Two independent tracks exist and neither requires a frozen upstream stage:

Tools

/api/tools

Gap Analyzer compares code against an SRS, FRS or user manual and reports the gaps; it has its own freeze/unfreeze and export. **Transformer** is a standalone code-transform super-agent (for example Helidon→Spring Boot, Oracle→PostgreSQL).

Multi-Agent CodeGen

stage 4

An agentic alternative to single-shot Stage-4 generation. It *does* require a frozen Architecture. Backend and tests only — the UI still drives the single-shot flow, so the page does not exercise this path.

02 Technology stack

Versions below are read from `backend/requirements.txt`, `frontend/package.json` and the `Dockerfile`. Treat them as pinned — do not bump them silently.

Backend

Component	Version	Role
Python	3.11-slim-bookworm	Runtime image
FastAPI	0.110.1	HTTP API, 21 router mounts
Uvicorn	0.25.0	ASGI server
Motor / PyMongo	3.3.1 / 4.5.0	Async MongoDB driver
Pydantic	2.13.4	Models and validation
httpx	0.28.1	All outbound LLM/provider calls
qdrant-client	1.18.0	Vector index (optional)
sentence-transformers	5.5.0	Local embeddings
PyGithub	2.9.1	Repository push
PyYAML	6.0.3	KB context export
python-jose · passlib	3.5.0 · 1.7.4	JWT and bcrypt
langgraph · langchain-core	0.2.60 · 0.3.29	Opt-in confidence engine
dulwich	>=1.2.5, <2.0	Pure-Python git fallback

Frontend

Component	Version	Role
React · React DOM	^19.0.0	UI runtime
react-router-dom	^7.5.1	Routing
CRA · CRACO	5.0.1 · ^7.1.0	Build toolchain
Tailwind CSS	^3.4.17	Styling, with Radix UI + shadcn/ui

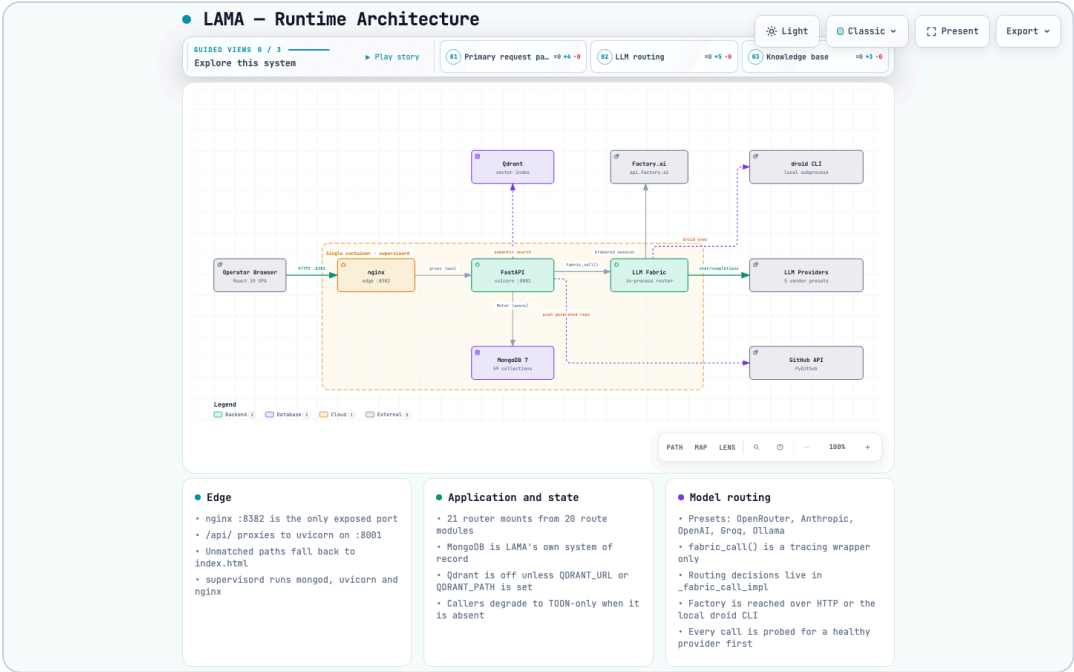
axios	<code>^1.8.4</code>	API client, 600 s timeout
D3 · Mermaid	<code>7.9.0</code> · <code>^11.15.0</code>	ER diagrams and rendered diagrams
Monaco Editor	<code>^4.7.0</code>	Generated-code editing
Recharts	<code>^3.6.0</code>	Admin analytics
react-resizable-panels	<code>2.1.7</code> <code>pinned</code>	Panel layout
yarn	<code>1.22.22</code>	Package manager — never npm

Data and models

Component	Role
MongoDB 7	LAMA's own system of record — 59 collections. PostgreSQL is the migration <i>target</i> , never LAMA's store.
Qdrant	Optional semantic index. Inert unless <code>QDRANT_URL</code> or <code>QDRANT_PATH</code> is set; callers degrade to TOON-only.
LLM providers	Five vendor presets — OpenRouter, Anthropic, OpenAI, Groq, Ollama — plus a <code>custom</code> scaffold.
Factory.ai	Optional per-project brokered execution, over HTTP API or the local <code>droid</code> CLI.

03 Runtime architecture

LAMA ships as a **single container**. `supervisord` runs three processes — `mongod`, `uvicorn` and `nginx` — and exposes exactly one port.



Runtime topology. Interactive version: [architecture/diagrams/lama-runtime.html](#)

Request path

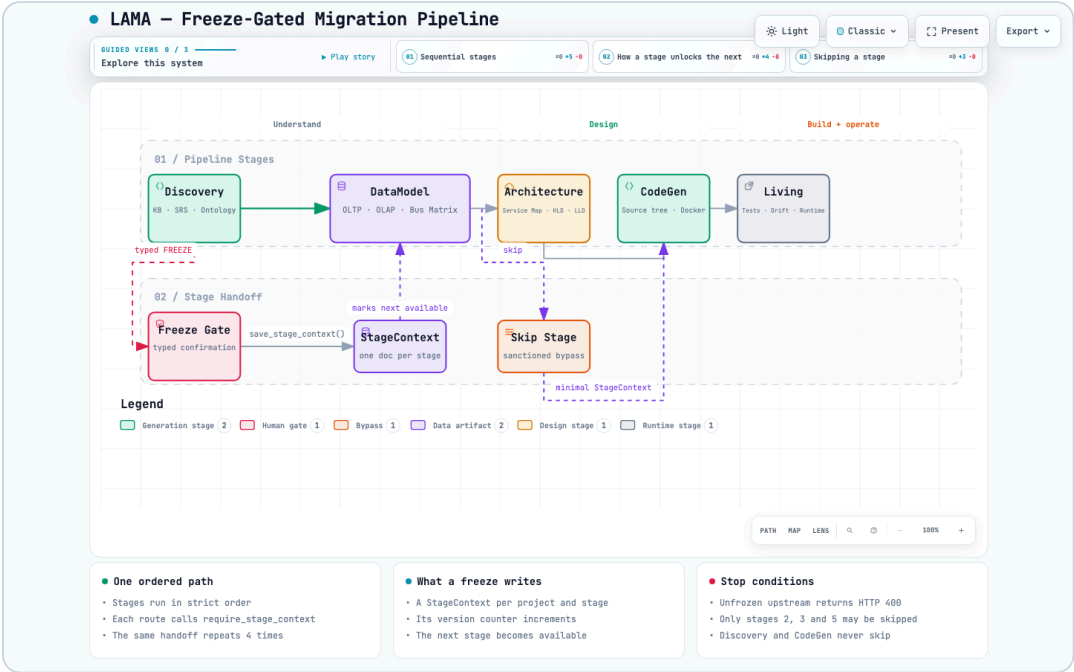
Hop	Behaviour
Browser → nginx	Port <code>8382</code> is the only exposed port.
nginx → FastAPI	<code>/api/</code> proxies to <code>127.0.0.1:8001</code> . <code>/health</code> maps to <code>/api/health</code> .
Static assets	Served from <code>/usr/share/nginx/html</code> ; unmatched paths fall back to <code>index.html</code> for the SPA.
FastAPI → Mongo	Motor async driver; the single source of truth for collection accessors is <code>backend/db.py</code> .
FastAPI → Qdrant	Optional. Off unless a URL or embedded path is configured.

nginx is tuned for this workload. `client_max_body_size 100M` for ZIP uploads, `proxy_buffering off` so SSE streams are not held back, and 600 s read/send timeouts for long generation jobs. The SPA shell is served `no-store`; hashed assets cache immutably for a week.

Architecture does not stream. Its long jobs run as background tasks polled every 2 s, because production Kubernetes ingress imposes a 60 s timeout. DataModel and SRS may stream — they emit short chunks.

04 The pipeline and its freeze gates

The pipeline is the spine of the product. Each stage is unlocked only by freezing the one before it, and a freeze is a deliberate, typed human action.



The five stages, the handoff that unlocks each one, and the skip bypass. Interactive version: [architecture/diagrams/lama-pipeline.html](#)

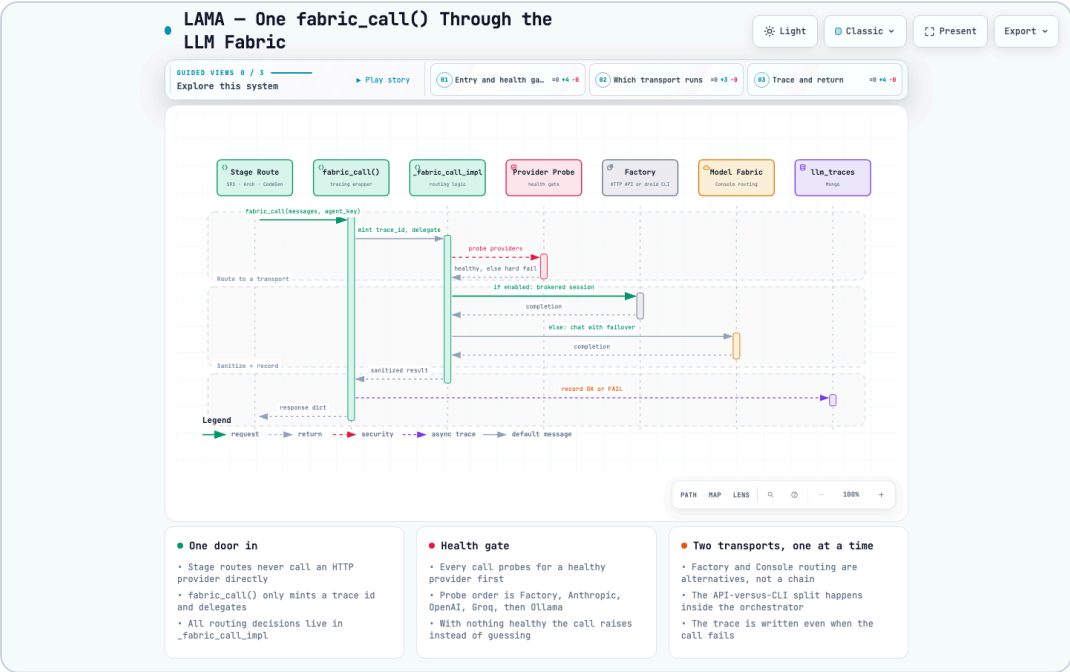
What a freeze actually does

#	Step	What happens
1	Operator types the word	Freeze and reset gates require a typed <code>FREEZE</code> / <code>RESET</code> in the UI. Intentional friction — do not weaken it.
2	Confidence engine votes	Two to three independent models score the artifact section by section against the knowledge base. Results land in <code>stage_confidence</code> as per-stage badges; responses that do not match the JSON schema are discarded from the vote.
3	StageContext is written	<code>save_stage_context()</code> upserts one document per <code>(project_id, stage)</code> and auto-increments its <code>version</code> .
4	Next stage is promoted	Promotion is implicit — the handler sets <code>project.stage_status[N+1] = "available"</code> .

Only intermediate stages may be skipped. DataModel, Architecture and Living can be marked skipped; **Discovery and CodeGen never can.** A skipped stage writes a minimal StageContext purely so downstream gates pass.

05 The LLM fabric

Every model call in the product goes through **one** entry point. Stage routes never call a provider over HTTP directly.



A single fabric_call() , from stage route to provider and back. Interactive version: [architecture/diagrams/lama-fabric-call.html](#)

Know which of the three modes is live before debugging a prompt. fabric_call() itself is only a tracing wrapper — it mints a trace id and delegates; every routing decision lives in _fabric_call_impl . Each call also probes for a healthy provider first (Factory, then Anthropic, OpenAI, Groq, Ollama) and raises rather than guessing if none is. Both outcomes reach llm_traces , so a failure is as traceable as a success.

Mode	When it runs
Console routing default	AGENT_COMPLEXITY[agent_key] picks a tier; provider.routing[tier] picks the model. Failover walks every active provider on quota or auth errors.
Factory API	Brokered to api.factory.ai when the project enables it. Poll-based; slow agents get a longer timeout and a shrink-and-retry path.
Factory CLI	LAMA_FACTORY_MODE=cli drives the local droid binary. The API-versus-CLI choice is made inside the orchestrator — from llm.py there is one Factory door.

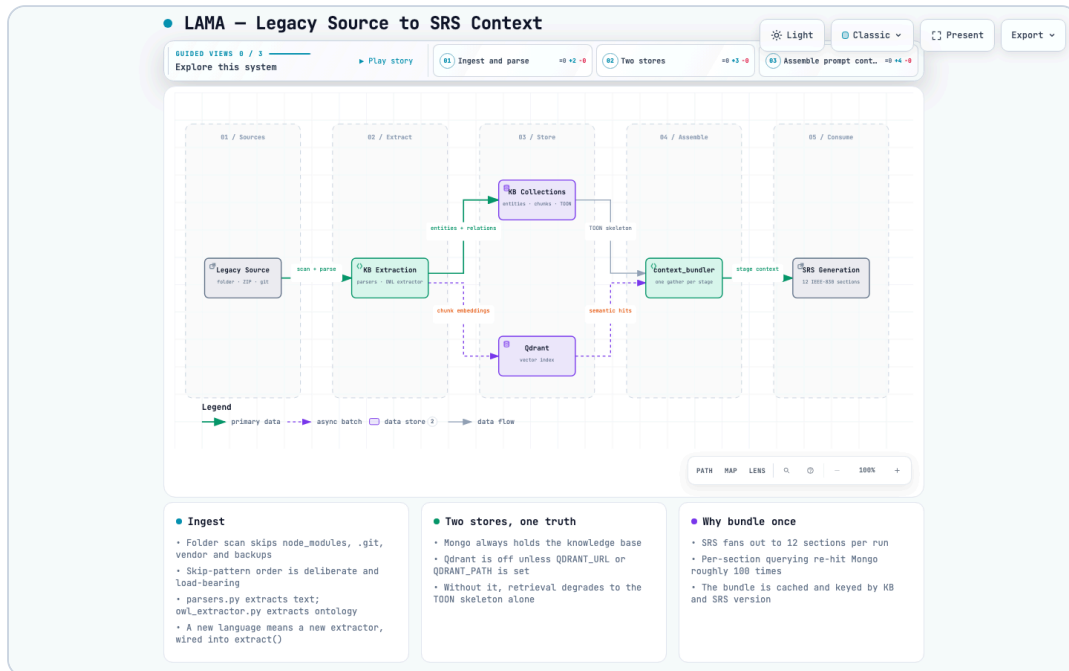
Complexity tiers

Each agent key carries a tier that the Console maps to a concrete model — no hard-coded vendor defaults at call sites.

- high** — srs.regenerate , datamodel.oltp , arch.recommend , arch.hld , codegen.planner , codegen.coder_be , codegen.coder_fe .
- medium** — srs.generate , arch.llld , codegen.verifier , codegenReviewer , codegenTester .
- low** — srs.gap_question , datamodel.bus_matrix , codegen.build_tool_selector , codegenFinalizer .

06 Knowledge base and Discovery

Discovery turns unstructured legacy source into typed, queryable context that every later stage consumes.



From legacy source to SRS prompt context. Interactive version:

<architecture/diagrams/lama-discovery-kb.html>

Ingestion

Source arrives as a folder scan, a ZIP upload or a git clone. The folder scanner deliberately skips `node_modules`, `.git`, `vendor`, `__pycache__` and backup patterns (`*.bak`, `*.save`, `*.bcp`, `*.old`, `*.backup`, `*.php*`). **The order of these patterns matters** — it fixed a real defect.

Extraction

`kb/parsers.py` extracts text per file type (PHP, SQL, PDF, DOCX, CSV, TXT, ZIP); `kb/owl_extractor.py` derives the ontology — classes, methods, tables, columns, relations, roles. It is **language-agnostic**: a new language means an `extract_<lang>` function wired into *both* `extract()` and `extract_zip()`.

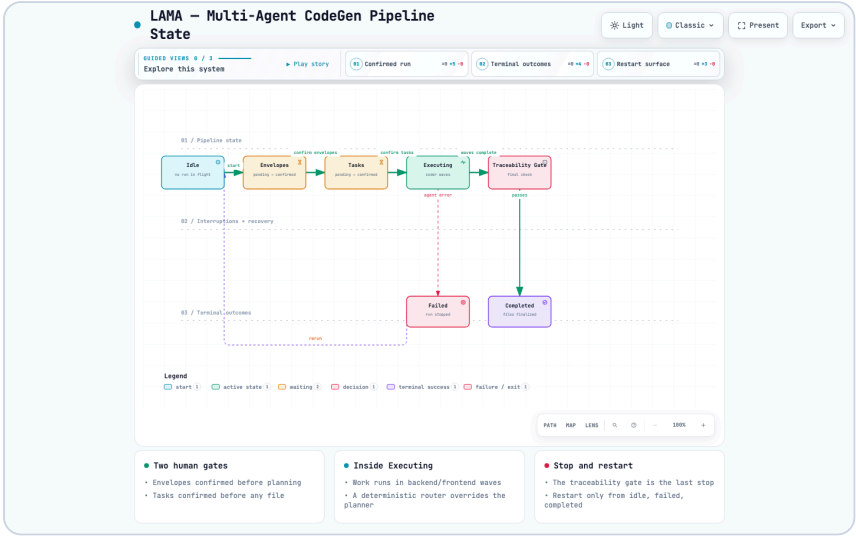
The SRS

Discovery produces an IEEE-830 style SRS in **12 sections**: `introduction`, `overall_description`, `actors_use_case_inventory`, `specific_requirements`, `detailed_use_cases`, `external_interfaces`, `non_functional_requirements`, `integration_requirements`, `validation_verification`, `traceability_matrix`, `appendices` and `entity_model`.

Build stage context once, not per section. `context_bundler.build_stage_context()` gathers everything a stage prompt needs in one pass and caches it, keyed by project, stage, KB version, SRS version and target stack — so a KB rebuild or SRS edit invalidates it automatically. SRS fans out to 12 sections; re-querying Mongo per section is the problem this was written to kill.

07 Multi-agent CodeGen

An agentic alternative to single-shot Stage-4 generation, driven by a state machine in one `codegen_pipeline_state` document per project. It requires a frozen Architecture.



Pipeline states and the two human confirmation gates. Interactive version: `architecture/diagrams/lama-codegen-multiagent.html`

The agent chain

`context_manager` → `build_tool_selector` → `planner` → `coder_be` | `coder_fe` → `verifier` → `reviewer` → `tester` → `traceability_gate` → `finalizer`, after a `super_agent` initialises the run. The coders draw a `high` tier model, the finalizer `low`.

BE/FE routing is deterministic, not advisory. The Planner proposes; `_route_task_to_coder()` then overrides from the target path. Unambiguous backend paths go to `coder_be`, unambiguous frontend paths to `coder_fe`; only truly ambiguous paths keep the Planner's choice. A frontend file must never reach the backend coder.

States

Status	Meaning
idle	No run in flight
envelopes_pending → envelopes_confirmed tasks_pending → tasks_confirmed	The two human gates — envelopes discovered, then tasks planned, each awaiting confirmation
executing	Coder waves, split backend/frontend
traceability_gate	Final check before finalize
completed · failed	Terminal — a run restarts only from idle, failed or completed

08 Data model

`backend/db.py` is the **single source of truth** for collections — 59 of them. A new collection is added there or it does not exist. Thirty-nine are project-scoped and indexed as such by `ensure_indexes()` at startup.

Group	Collections
Core & Discovery	<code>projects</code> , <code>kb_files</code> , <code>kb_chunks</code> , <code>kb_entities</code> , <code>kb_toon</code> , <code>kb_deep_analysis</code> , <code>conversations</code> , <code>messages</code> , <code>srs_documents</code> , <code>prompts</code> , <code>project_prompts</code> , <code>freeze_gates</code> , <code>audit_log</code> , <code>stage_context</code>
Stage 2 · DataModel	<code>data_models</code> , <code>bus_matrix</code> , <code>olap_models</code> , <code>migration_artifacts</code>
Stage 3 · Architecture	<code>arch_documents</code> , <code>arch_services</code>
Stage 4 · CodeGen	<code>codegen_files</code> , <code>codegen_runs</code> , <code>parity_runs</code>
Stage 5 · Living	<code>living_artifacts</code> , <code>living_runs</code> , <code>ontology_snapshots</code> , <code>business_ontologies</code>
Multi-agent CodeGen	<code>codegen_envelopes</code> , <code>codegen_tasks</code> , <code>codegen_agent_runs</code> , <code>codegen_pipeline_state</code>
Transformer	<code>transformer_envelopes</code> , <code>transformer_tasks</code> , <code>transformer_agent_runs</code> , <code>transformer_agent_configs</code>
Tools	<code>gap_analyses</code> , <code>gap_analysis_files</code> , <code>transformations</code> , <code>transform_files</code> , <code>tools_kb</code>
Graph & journeys	<code>legacy_analysis</code> , <code>kb_graph</code> , <code>kb_journeys</code> , <code>kb_journey_config</code>
Console	<code>model_providers</code> , <code>agent_configs</code> , <code>token_usage_log</code>
Multi-tenancy	<code>tenants</code> , <code>users</code>
Confidence	<code>stage_confidence</code> , <code>living_reports</code>
Sessions & tracing	<code>agent_sessions</code> , <code>llm_traces</code>
Other	<code>github_configs</code> , <code>data_sources</code> , <code>project_integrations</code> , <code>kb_git_sources</code> , <code>kb_module_selection</code> , <code>factory_workspaces</code>

Multi-tenant since iter-13.68. JWT bearer auth resolves a token to `{user, tenant}`; a super-admin carries `tenant_id == "*"` . Any new route that reads project data must scope by tenant, not by `project_id` alone.

09 API surface

Everything mounts under `/api`. There are **21 router mounts** from **20 route modules** — `routes/datamodel.py` contributes two. One file per route concern; modules do not cross-import, they go through `pipeline.py`.

Prefix	Module	Concern
<code>/projects</code>	<code>projects.py</code>	Project CRUD, tenant-scoped
<code>/projects</code>	<code>datamodel.py</code>	Factory orchestrator config <code>2nd mount</code>
<code>/kb</code>	<code>kb.py</code>	Upload, scan, build, clone <code>shared</code>
<code>/kb</code>	<code>db_ingest.py</code>	Live DB + app-URL ingestion <code>shared</code>
<code>/srs</code>	<code>srs.py</code>	SRS generate, edit, freeze
<code>/data-model</code>	<code>datamodel.py</code>	Stage 2 artifacts
<code>/architecture</code>	<code>architecture.py</code>	Stage 3 jobs and artifacts
<code>/codegen</code>	<code>codegen.py</code>	Stage 4, incl. multi-agent
<code>/living</code>	<code>living.py</code>	Stage 5
<code>/pipeline</code>	<code>pipeline.py</code>	Skip/unskip, confidence
<code>/tools</code>	<code>tools.py</code>	Gap Analyzer + Transformer
<code>/console</code>	<code>console.py</code>	Providers, agents, usage
<code>/chat</code>	<code>chat.py</code>	TOON-context chat
<code>/sessions</code>	<code>sessions.py</code>	Rolling agent memory
<code>/context</code>	<code>context.py</code>	Context-bundle inspection
<code>/auth</code> · <code>/admin</code>	<code>auth.py</code> · <code>admin.py</code>	Login; tenant/user CRUD
<code>/prompts</code> · <code>/audit</code>	<code>prompts.py</code> · <code>audit.py</code>	Prompt library; audit trail
<code>/github</code> · <code>/integrations</code>	<code>github.py</code> · <code>integrations.py</code>	Push config; integration catalog

Two prefixes are intentionally shared. `/kb` is served by both `kb.py` and `db_ingest.py`, and `/projects` by both `projects.py` and `datamodel.py`. This is by design, not a collision to "fix".

On the frontend, **every** backend call lives in `frontend/src/lib/api.js`. A request interceptor attaches the bearer token; a response interceptor clears credentials and redirects to `/login` on HTTP 401.

10 Deployment and operations

Local development — split processes

TERMINAL · fast iteration

```
# backend — hot reload on :8000
$ cd backend && uvicorn server:app --reload --port 8000

# frontend — CRA dev server on :3000
$ cd frontend && yarn install && yarn start
```

Single image — production-like

DOCKER · port 8382

```
$ docker compose up -d
$ docker compose logs -f lama
$ curl http://127.0.0.1:8382/health      # → {"ok":true}

# after Python edits
$ docker compose restart lama

# after React edits — nginx serves the bind-mounted build
$ (cd frontend && yarn build) && docker compose restart lama
```

Boot order

`supervisord` starts three processes by priority: **10** `mongod` (bound to `127.0.0.1:27017`, data in `/data/db`), **20** `uvicorn` (`server:app` on `:8001`, proxy headers trusted), and **30** `nginx`. On every backend start a FastAPI startup event runs `run_seed()` — idempotent and safe on each boot — seeding the prompt library, the PMIS pilot, agent configs, tier migrations and default tenancy, then building indexes.

Tests

```
$ pytest backend/tests/                # ~70 suites
$ pytest backend/tests/test_lama_v4.py -k test_chat -x
$ pyflakes backend/routes/codegen.py   # standing verification step
```

Suites are named `test_iter<N>_<topic>.py` and pin **one** iteration's behaviour — read the matching `memory/PRD.md` entry first. A `502` on chat-edit tests is acceptable; they assert `in (200, 502)`.

11 Configuration

Variable	Purpose
MONGO_URL · DB_NAME	Required. System of record.
OPENROUTER_API_KEY	Needed for any meaningful LLM call via the env fallback path.
LAMA_DEFAULT_MODEL	Deliberately empty — the Console's routing table picks per agent key.
QDRANT_URL · QDRANT_API_KEY	Remote vector index. Optional.
QDRANT_PATH	Embedded on-disk vector engine — no server needed.
LAMA_JWT_SECRET	HS256 signing key. Set it in production.
LAMA_SESSION_SIGNING_KEY	HMAC for agent-memory summaries. A mismatch wipes the summary by design.
LAMA_FACTORY_MODE	cli drives the local droid binary.
FACTORY_API_BASE_URL	Defaults to https://api.factory.ai/api/v0 .
LAMA_DISABLE_OPENROUTER_FALLBACK	1 stops silent fallback spend.
LAMA_CONFIDENCE_ENGINE	langgraph selects the opt-in alternative engine.
LAMA_BR_ENFORCE · LAMA_BR_MIN_COVERAGE	Block freeze when business-rule coverage is below threshold.
CORS_ORIGINS	Wide open by default. Tighten in production.
LAMA_DISABLE_SSL_VERIFY · LAMA_CA_BUNDLE	For corporate TLS proxies.

Secrets hygiene. backend/ .env holds real API keys and is gitignored; it is deleted from the image at build time so production configuration comes from -e flags or compose environment: . The root .env is committed and must carry only non-secret compose defaults. Keep that split.

12 Conventions and known footguns

Rules of engagement

Rule	Why
Read <code>memory/PRD.md</code> first	Append-only iteration log; recent entries override earlier ones and the file is always newer than the summaries.
Never invent stages, collections or routes	New collection → <code>db.py</code> . New route → register in <code>server.py</code> . New API call → <code>lib/api.js</code> .
All LLM calls via <code>fabric_call()</code>	Never <code>httpx</code> directly — it bypasses routing, tracing and budgets.
Never run <code>npm install</code>	yarn-only, via corepack.
<code>data-testid</code> is part of the contract	The testing agent asserts on them; keep them stable.
Audit-log state changes	Via the <code>audit_log</code> collection.
Pydantic over raw <code>dict</code> bodies	For any new POST payload.

Footguns

Trap	What to do
Python version skew	The image is 3.11 but a local venv may be 3.14. Reproduce version-sensitive bugs inside Docker, not the local venv.
Compose masks the image	<code>./frontend/build</code> and <code>./backend</code> are bind-mounted over the baked-in copies. A stale local build silently serves old UI even after pulling a new image.
Compose overrides the entrypoint	It routes through a wheels-install wrapper, so debugging boot means reading that script, not <code>docker/entrypoint.sh</code> .
Inactive provider, valid flag	A provider marked <code>is_active</code> with an invalid key produces a cascade of 401s. A validate-ping endpoint is still backlog.
CodeGen file tree is flattened	Non-recursive on purpose, due to a babel plugin defect. Do not "fix" by recursing without testing.
SRS auto-trigger swallows failures	<code>srs_triggered</code> failures are silent. Known backlog item.
Markdown fences from the model	Handled by <code>SRSPanel.jsx::normaliseSectionContent</code> . Reuse it on any new render path.

Where to look

Goal	Open
Understand the pipeline	<code>backend/pipeline.py</code> + <code>memory/PRD.md</code>
Find which collection stores X	<code>backend/db.py</code>
Debug which model actually ran	<code>llm.py::fabric_call</code> → <code>llm_traces</code> , <code>token_usage_log</code>
Gather context for a stage prompt	<code>backend/context_bundler.py</code>
Add an LLM provider	<code>backend/fabric/model_fabric.py</code> → <code>PROVIDER_PRESETS</code>
Add an SRS section	<code>backend/routes/srs.py</code> → <code>SECTION_CONFIGS</code>
Deep architecture narrative	<code>docs/ARCHITECTURE.md</code> <code>see note</code>

Documentation currency. `docs/ARCHITECTURE.md` self-declares iteration 14.27 while `memory/PRD.md` has reached 17.18 — roughly three major iterations of drift. Its own header defers to `memory/PRD.md` on conflict. Prefer the PRD, then the code.

About this handbook. Every version number, count and contract here was read from the source tree, not from prose. The five diagrams were generated with Archify from typed sources in `architecture/src/` and verified against the code; the audit is in `architecture/report.md`.